



Organisation nationale de lutte contre le
faux-monnayage (ONCFM)

Contexte du projet

- mettre en place un algorithme qui soit capable de différencier automatiquement les vrais des faux billets
- des différences de dimensions entre les vrais et les faux billets difficilement visibles à l'œil nu
- construire un algorithme à partir des caractéristiques géométriques d'un billet

Données pour la construction de l'algorithme

- billets.csv : **1000 vrai** et **500 faux**
- six informations géométriques sur un billet :
 - **length** : la longueur du billet (en mm)
 - **height_left** : la hauteur du billet (mesurée sur le côté gauche, en mm) ;
 - **height_right** : la hauteur du billet (mesurée sur le côté droit, en mm) ;
 - **margin_up** : la marge entre le bord supérieur du billet et l'image de celui-ci (en mm) ;
 - **margin_low** : la marge entre le bord inférieur du billet et l'image de celui-ci (en mm) ;
 - **diagonal** : la diagonale du billet (en mm)

Analyse exploratoire

Billets vrais (lot de 1000)

Moyenne (en millimètres):

- diagonal : 171.98
- height_left : 103.94
- height_right : 103.80
- margin_low : 4.11 (29 valeurs nulles)
- margin_up : 3.05
- length : 113.20



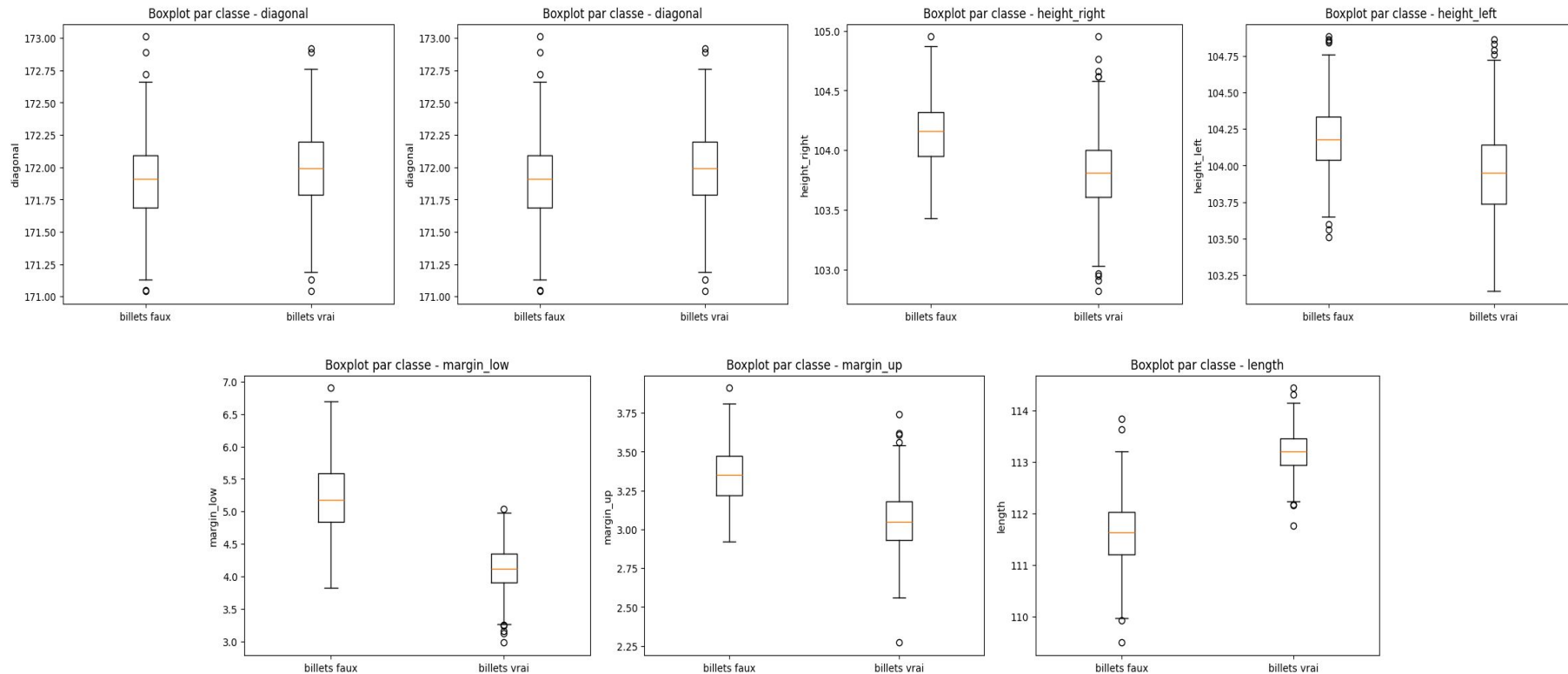
Billets faux (lot de 500)

Moyenne (en millimètres):

- diagonal : 171.90 (-0.08)
- height_left : 104.19 (+0.25)
- height_right : 104.14 (-0.34)
- margin_low : 5.21 (+1.1) (8 valeurs nulles)
- margin_up : 3.35 (+ 0.3)
- length : 111.63 ()



Analyse exploratoire



Régression linéaire

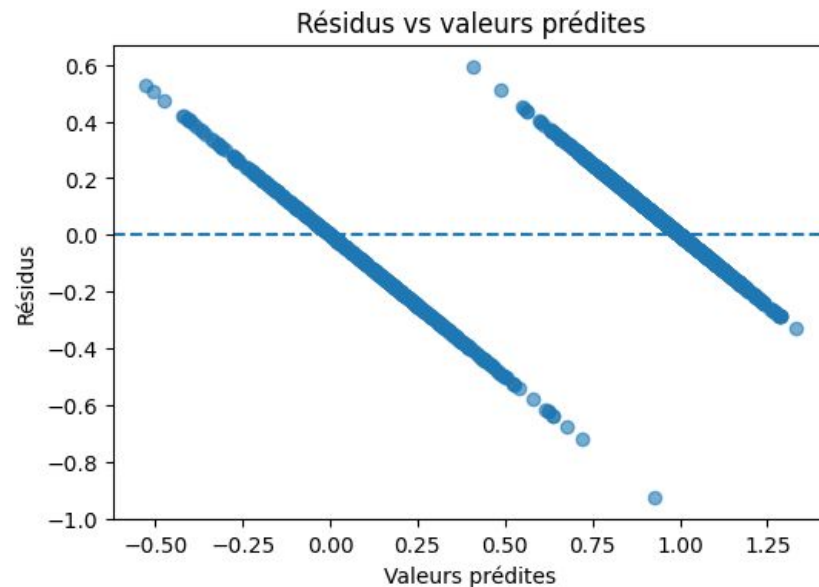
- Approche statistique pour prédire une valeur numérique
- colonne “**margin_low**” avec **37 valeurs nulles**
- **But** : prédire une valeur numérique (margin_low) à partir de mesures (ex. diagonal, margin_up, ...)
- **Modèle** : on ajuste une droite/plan : $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$
- **Apprentissage** : choisir les β qui minimisent la somme des carrés des erreurs (méthode des moindres carré).

Régression linéaire

Vérification d'application :

- La linéarité :
 - le nuage de résidus montre deux relations différentes (vrai/faux)
- L'indépendance des erreurs
 - Durbin–Watson = 1,682 → légère autocorrélation positive
- La Variance constante (homoscédasticité)
 - Breusch–Pagan : p-value = 0,0000

Dans notre cas : nous pouvons utiliser la régression linéaire.



Régression linéaire

- **Données incomplètes** : margin_low a des valeurs manquantes → on impute (ex. médiane) avant d'entraîner
- Pipeline scikit-learn = prétraitement (SimpleImputer sur margin_low) + modèle (LinearRegression) → propre et sans fuite train/test
- **Évaluation** :
 - **R²** : part de la variabilité expliquée (peut être négatif si le modèle fait pire qu'une moyenne).
 - **RMSE** : erreur moyenne en unités de la cible (plus petit = mieux).

Application de la régression linéaire

- **R^2 CV (5-fold) = $-0,9263$, écart-type = $0,5603$**
- Analyse :
 - On a fait une validation croisée à **5 plis** :
 - On entraîne sur 4/5 des données et on évalue R^2 sur le 1/5 restant puis on répète 5 fois.
 - On affiche ensuite la moyenne des 5 scores et leur écart-type (d'où " $\pm 0,5603$ ")
 - $R^2 < 0$ signifie que le modèle prédit moins bien que la moyenne constante de margin_low
- **Conclusion** : Une moyenne à $-0,9263$ indique **une sous-performance** nette face au baseline "moyenne". L'écart-type $0,5603$ montre **des scores très instables** selon le pli

Extrait des 10 premiers billet avec margin_low manquant

index	is_genuine	diagonal	height_left	height_right	margin_low	margin_up	length
72	True	171.94	103.89	103.45	4.275320	3.25	112.79
99	True	171.93	104.07	104.18	4.629997	3.14	113.08
151	True	172.07	103.80	104.38	4.499841	3.02	112.93
197	True	171.45	103.66	103.80	4.794650	3.62	113.27
241	True	171.83	104.14	104.06	4.507956	3.02	112.36
251	True	171.80	103.26	102.82	3.473371	2.95	113.22
284	True	171.92	103.83	103.76	4.401298	3.23	113.29
334	True	171.85	103.70	103.96	4.268823	3.00	113.36
410	True	172.56	103.72	103.51	4.005235	3.12	112.95
413	True	172.30	103.66	103.50	4.064349	3.16	112.95

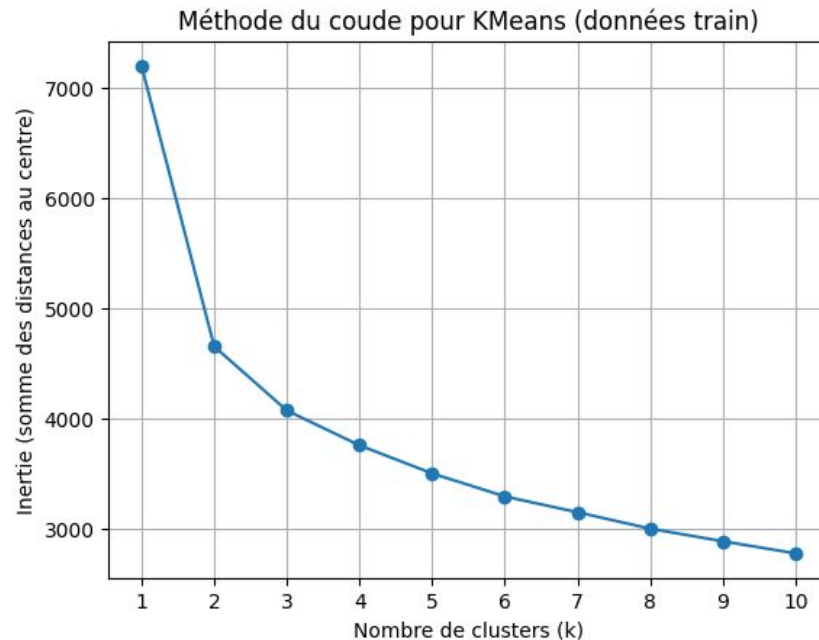
Différentes approches pour analyser les données

1. K-means
2. Régression logistique
3. KNN
4. Random Forest

Analyse via le K-means / Clustering

Le K-Means : mise en place

- Volumes : **1 200 en train** et **300 en test** (split 80/20).
- KMeans (k=2) a convergé en 5 itérations — normal/rapide.
- Alignement clusters → classes : fait sur le train par majorité ($\{0 \rightarrow 0, 1 \rightarrow 1\}$), puis appliqué au test.
- Matrices de contingence / confusion



Le K-Means : phase d'entraînement

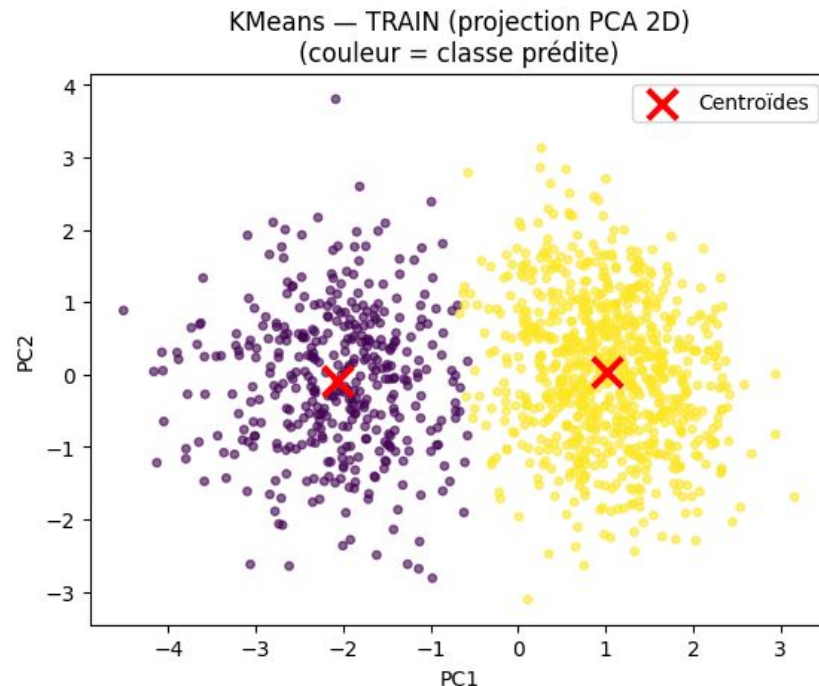
- Vrais 0 (faux billets) : 388 bien classés, 12 mal classés
→ rappel 0 = $388/400 = 0,970$.
- Vrais 1 (vrais billets) : 794 bien classés, 6 mal classés
→ rappel 1 = $794/800 = 0,993$.
- Prédits 0 : 394 au total, dont 388 corrects
→ précision 0 = $388/394 = 0,985$.
- Prédits 1 : 806 au total, dont 794 corrects
→ précision 1 = $794/806 \approx 0,985$.
- Accuracy train = **0,985** (12 + 6 erreurs sur 1 200).

Le K-Means : phase d'entraînement

- Matrice de contingence de l'entraînement

Cluster	Faux	Vrai
Billets faux	388	12
Billets vrais	6	794

- Répartition prédite
 - 394 faux billets
 - 806 vrais billets
- Volume de billets
 - 400 faux billets
 - 800 vrais billets



Le K-Means : phase de test

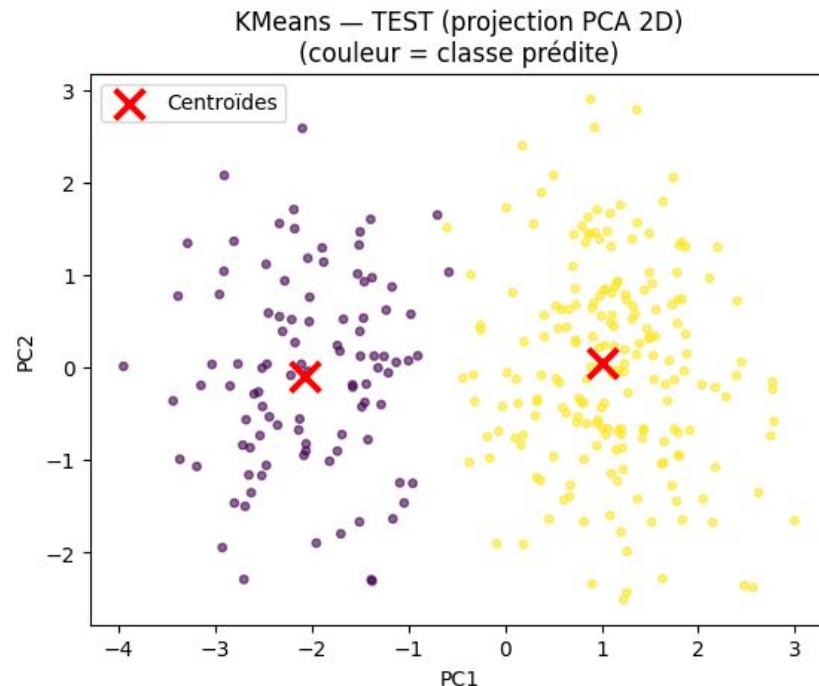
- Vrais 0 : 98 bien classés, 2 erreurs
→ rappel 0 = 0,98.
- Vrais 1 : 198 bien classés, 2 erreurs
→ rappel 1 = 0,99.
- Prédits 0 : 100 au total
→ précision 0 = $98/100 = 0,98$.
- Prédits 1 : 200 au total
→ précision 1 = $198/200 = 0,99$.
- Accuracy test = 0,987 (4 erreurs sur 300).

Le K-Means : phase de test

- Matrice de contingence de test

Cluster	Faux	Vrai
Billets faux	98	2
Billets vrais	2	198

- Répartition prédite
 - 100 faux billets
 - 200 vrais billets
- Volume de billets
 - 100 faux billets
 - 200 vrais billets



Le K-Means : conclusion

- KMeans sépare très bien : **~98–99 % de précision/rappel par classe**
- scatter-plots PCA montrent **deux amas bien distincts** : visuellement cohérent avec les métriques
- un algorithme excellent qui sépare bien **les vrais billet** et **les faux billets**
- Les rares erreurs (12/6 sur train, 2/2 sur test) sont probablement des cas limites proches de la frontière entre amas

Analyse via la régression logistique

La régression logistique : mise en place

La régression logistique donne une probabilité que le billet soit “vrai” (classe 1). On coupe à 0,5 pour dire “vrai” ou “faux”.

Les matrice de confusion comparent ce qui est vraiment (lignes) à ce que le modèle prédit (colonnes). Plus la diagonale est haute, mieux c’est.

Les courbes ROC et Precision–Recall montrent la qualité du modèle pour tous les seuils possibles (pas seulement 0,5). Plus elles sont en haut, mieux c’est.

La régression logistique : mise en place

- Volumes : **1 200 en train** et **300 en test**
- Convergence : le solveur s'est arrêté en 10 itérations — normal.
- Avec le seuil 0,5 (ce que signifie la matrice)

La régression logistique : phase d'entraînement

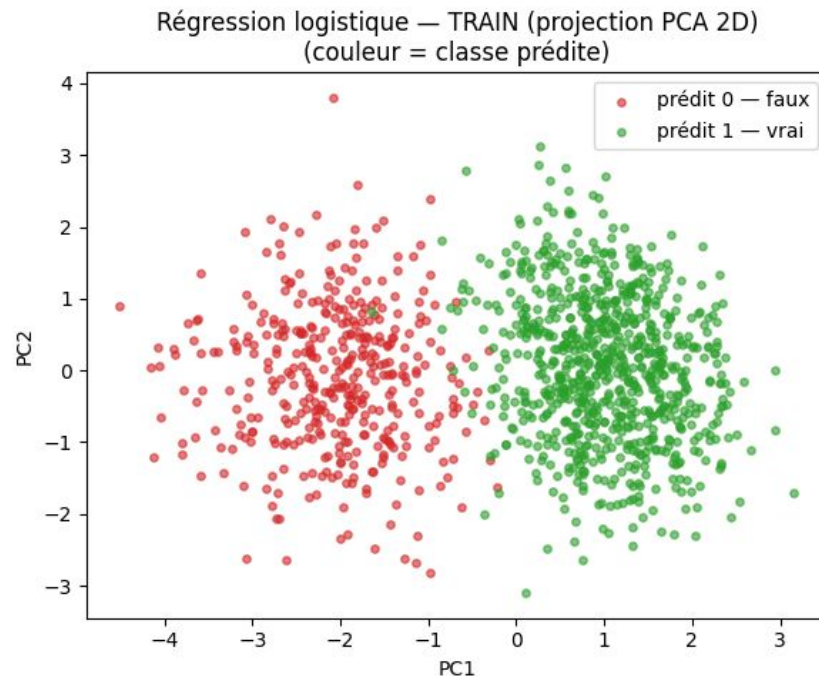
- **393 faux** correctement détectés, **7 faux pris pour des vrais**.
- **798 vrais** correctement détectés, **2 vrais pris pour des faux**.
- Accuracy train = **0,993**

La régression logistique : phase d'entraînement

- Matrice de contingence :

Cluster	Faux	Vrai
Billets faux	393	7
Billets vrais	2	798

- Répartition prédite
 - 395 faux billets
 - 805 vrais billets
- Volume de billets
 - 400 faux billets
 - 800 vrais billets



La régression logistique : phase de test

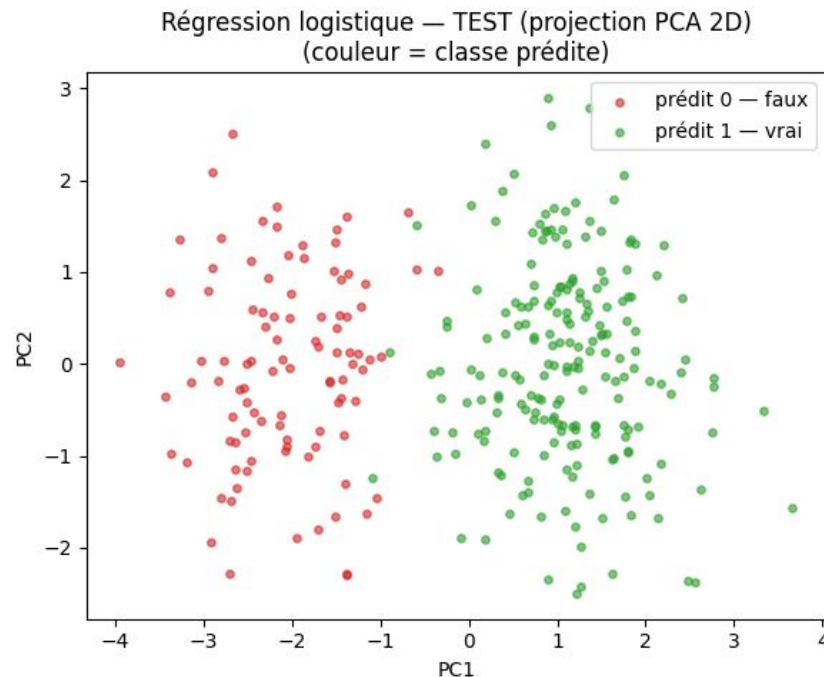
- **98 faux** correctement détectés, **2 faux pris pour des vrais**.
- **199 vrais** correctement détectés, **1 vrai pris pour un faux**.
- Accuracy test = **0,990** $\Rightarrow$ 3 erreurs sur 300 (excellent et très proche de la phase d'entraînement).

La régression logistique : phase de test

- Matrice de contingence :

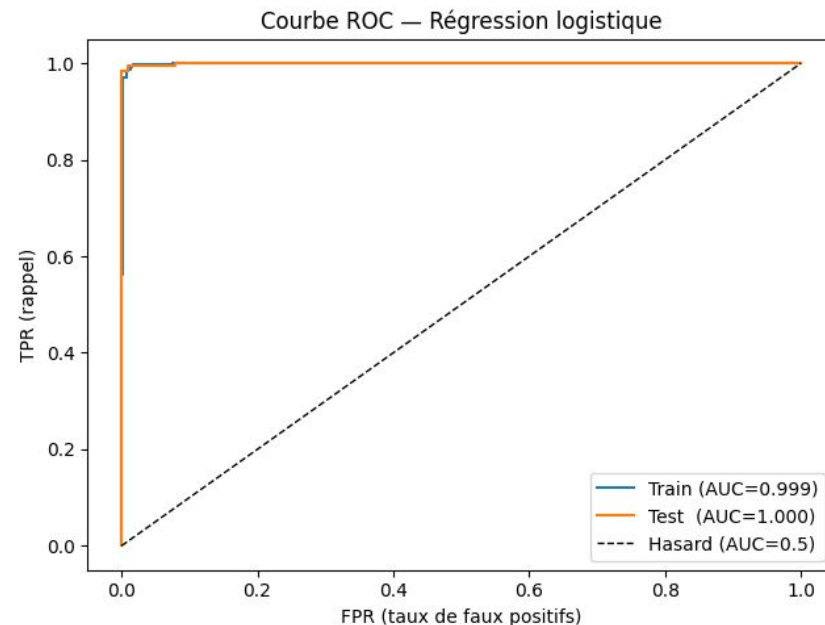
Cluster	Faux	Vrai
Billets faux	98	2
Billets vrais	1	199

- Répartition prédite
 - 99 faux billets
 - 201 vrais billets
- Volume de billets
 - 100 faux billets
 - 200 vrais billets



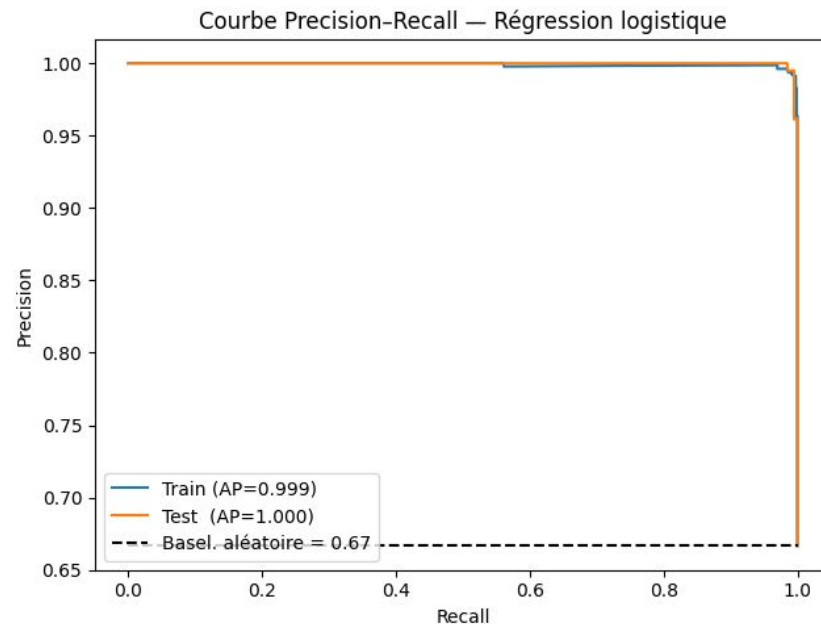
La régression logistique : courbe ROC

- Rappel :
 - x = faux positifs
 - y = vrais positifs
- Courbe roche de 1, donc la régression linéaire sépare **parfaitement les vrais et faux billets**



La régression logistique : courbe PR

- Si je veux beaucoup de vrais détectés (rappel), quelle sera ma précision ?
- Courbe PR: proche de 1, donc très peu de faux positif, sur une large plage de seuil.



La régression logistique : conclusion

- Garder le seuil à 0,5 donne **99,30% de prédiction** en phase de test en laissant passer **1 billet faux** sur un volume de 300 billets (**100 billets faux** et **200 billet vrais**)

Analyse via le KNN

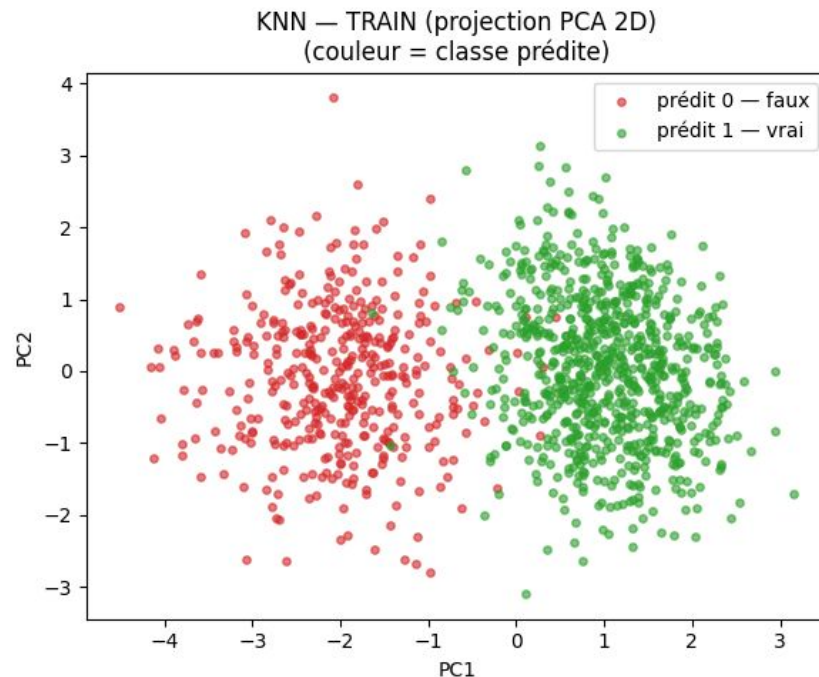
Le KNN : mise en place

- **Principe KNN** : classe un billet selon ses k plus proches voisins (après standardisation des variables).
- **Sortie** : proportion de voisins en classe 1 (“vrai”).
- **Décision** : seuil par défaut à 0,5 pour prédire 0/1 ; les courbes ROC et Precision–Recall évaluent la qualité pour tous les seuils.
- **Sensibilités** : choix de k, pondération par la distance, métrique de distance, et mise à l’échelle des variables.
- Volumes : **1 200 en train** et **300 en test**

Le KNN : phase d'entraînement

- 400 faux correctement détectés (TN), 0 faux pris pour des vrais (FP=0).
- 800 vrais correctement détectés (TP), 0 vrai pris pour un faux (FN=0).
- Accuracy train = 1,000 → zéro erreur sur train.
- Matrice de contingence :

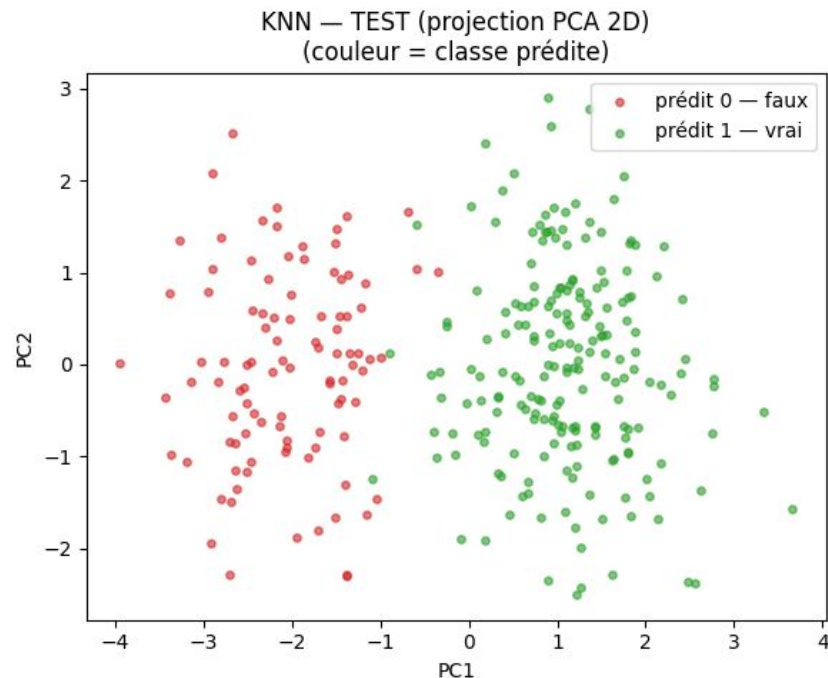
Cluster	Faux	Vrai
Billets faux	400	0
Billets vrais	0	800



Le KNN : phase de test

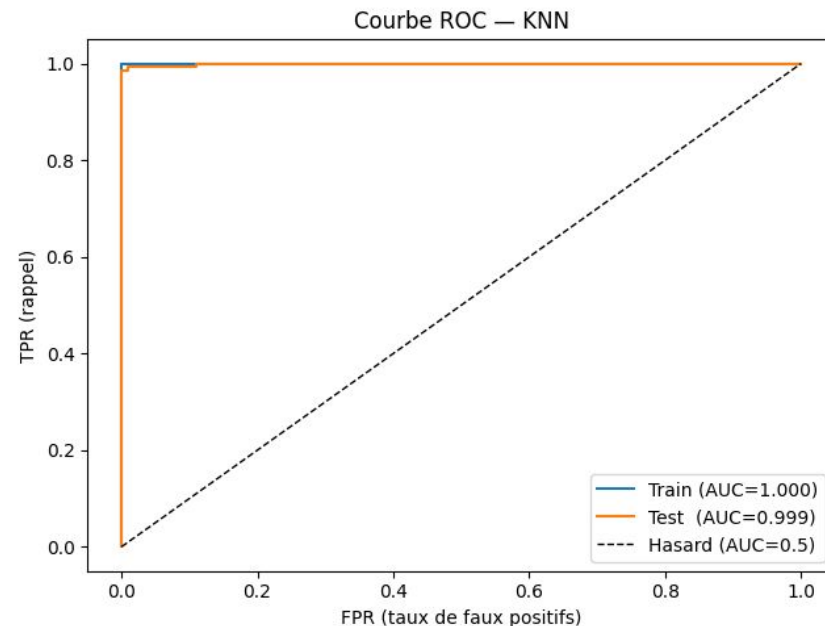
- 98 faux correctement détectés (TN), 2 faux pris pour des vrais (FP=2).
- 199 vrais correctement détectés (TP), 1 vrai pris pour un faux (FN=1).
- Accuracy test = 0,990 → 3 erreurs sur 300, identique à la logistique.
- Matrice de contingence :

Cluster	Faux	Vrai
Billets faux	98	2
Billets vrais	1	199



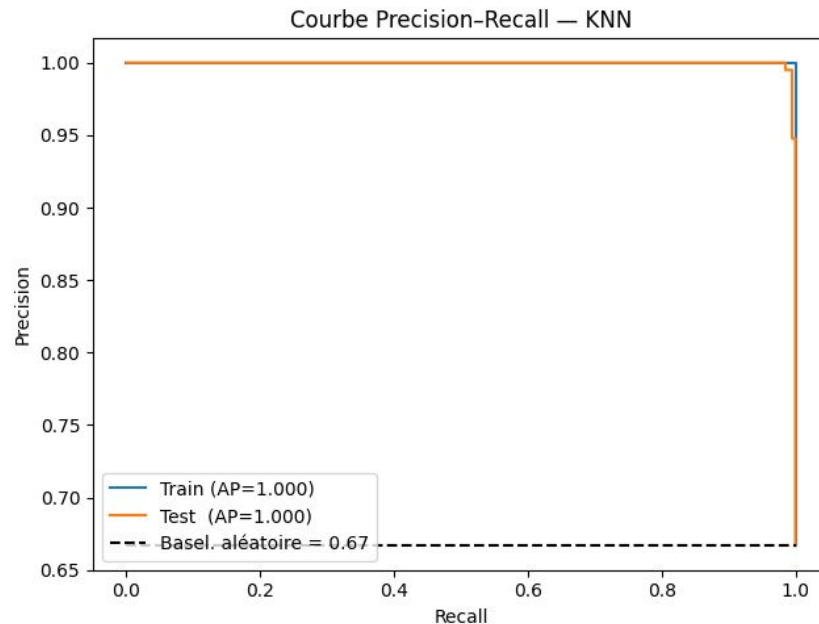
Le KNN : la courbe ROC

- La courbe est collée en haut à gauche.
- La courbe est très proche de 1, indiquant une capacité quasi parfaite du modèle à séparer les vrais des faux billets, quel que soit le seuil de décision choisi.



Le KNN : la courbe PR

- La courbe est quasi en haut à droite.
- La courbe PR est très proche de 1, montrant un excellent compromis entre précision et rappel sur une large plage de seuils.



Le KNN : conclusion

- **Performance exceptionnelle** : 99% de précision sur les données de test, un résultat équivalent à la régression logistique.
- **Mémorisation sans pénalité** : Bien qu'il "mémorise" parfaitement les données d'entraînement (100% de précision), cela n'impacte pas sa capacité à généraliser sur de nouvelles données.
- **Alternative robuste** : Le KNN est confirmé comme une solution fiable et performante pour ce problème de classification de billets.

Analyse via le Random Forest

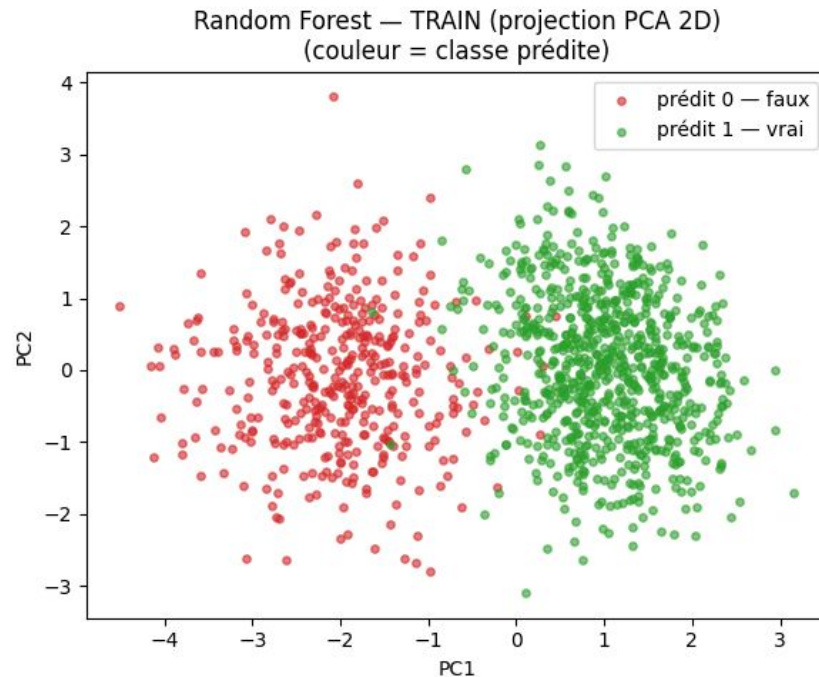
Le Random Forest : la mise en place

- Un ensemble d'arbres de décision entraînés sur des sous-échantillons (bagging) avec tirage aléatoire de variables à chaque split ; on moyenne/voit les prédictions pour gagner en précision et réduire l'overfitting
- Très performant “out-of-the-box” sur données tabulaires, peu de pré-traitements (généralement pas de mise à l'échelle requise, les arbres décident par seuils plutôt que par distances)
- La forêt renvoie des probabilités (moyenne des arbres) pour tracer ROC/PR et choisir un seuil métier ; on peut aussi obtenir une estimation OOB (out-of-bag) sans refaire des splits.
- Volumes : **1 200 en train** et **300 en test**

Le Random Forest : phase d'entraînement

- 400 faux correctement détectés, 0 faux pris pour des vrais
- 800 vrais correctement détectés, 0 vrai pris pour un faux
- Accuracy train = 1,000
- Matrice de contingence :

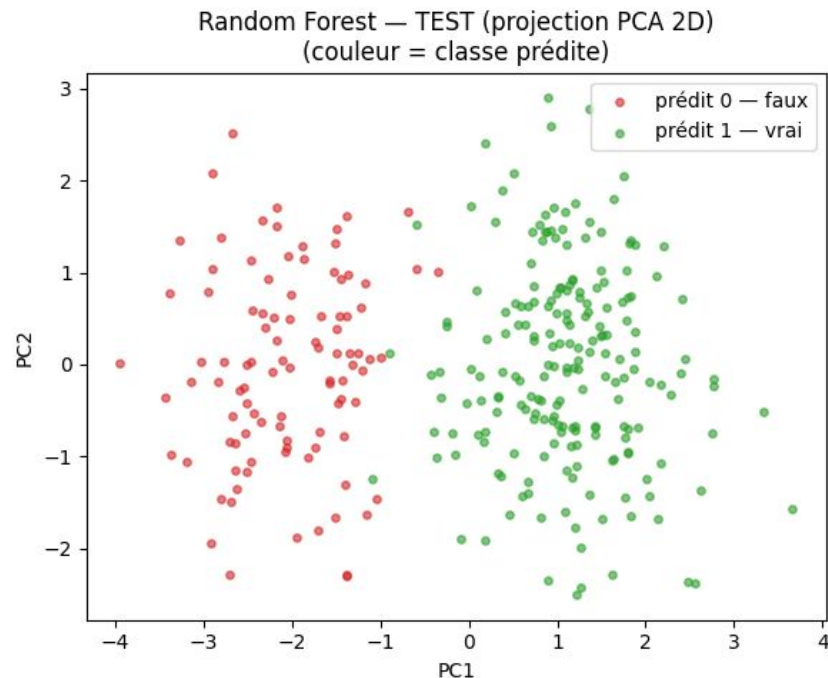
Cluster	Faux	Vrai
Billets faux	400	0
Billets vrais	0	800



Le Random Forest: phase de test

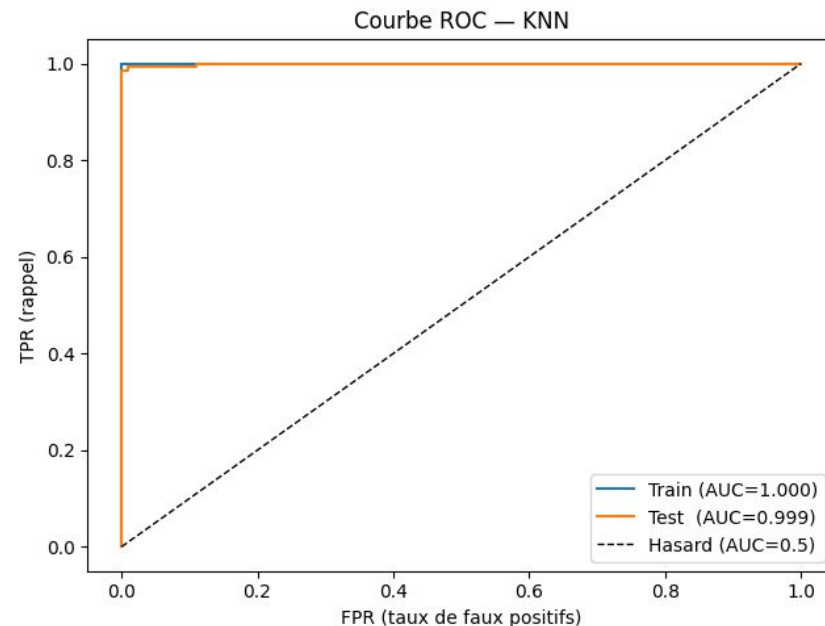
- 98 faux correctement détectés, 2 faux pris pour des vrais
- 199 vrais correctement détectés, 1 vrai pris pour un faux
- Accuracy test = 0,990 (3 erreurs sur 300).
- Matrice de contingence :

Cluster	Faux	Vrai
Billets faux	98	2
Billets vrais	1	199



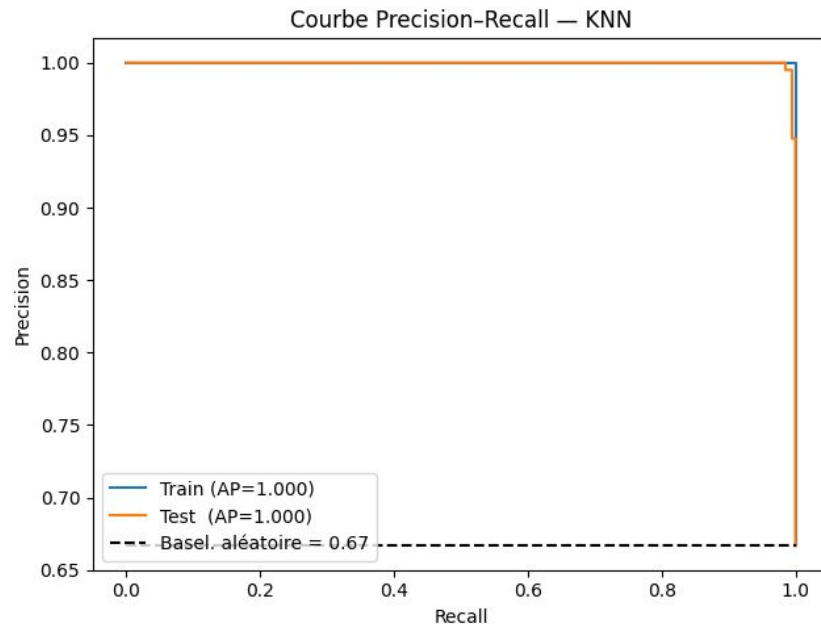
Le Random Forest : la courbe ROC

- La courbe ROC longe la zone en haut à gauche, proche du point idéal
- L'AUC est proche de 1, signe d'une séparation quasi parfaite entre vrais et faux billets, indépendamment du seuil choisi.



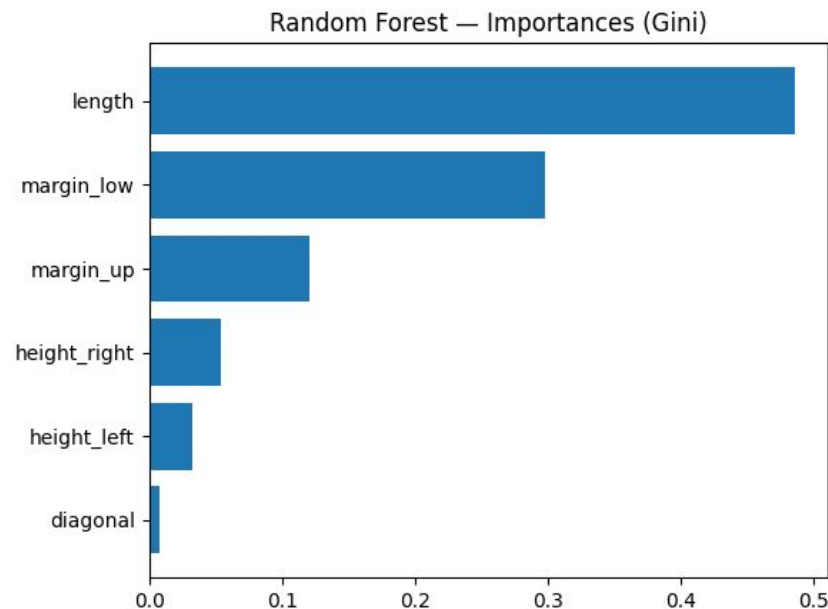
Le Random Forest: la courbe PR

- La courbe Precision–Recall reste au plus près du coin haut-droite, signe que précision et rappel demeurent élevés simultanément sur la plupart des seuils.
- L’Average Precision (AP) est proche de 1 : le modèle conserve une excellente précision tout en retrouvant la grande majorité des positifs sur une large plage de seuils.



Le Random Forest : importances (Gini)

- Impureté de Gini (dans un nœud) : mesure le mélange des classes.
- Plus la baisse d'impureté de Gini cumulée est grande, plus la variable est jugée "importante".)
- Analyse
 - **length (0,486)** : principal moteur des décisions de la forêt (plus forte baisse d'impureté).
 - **margin_low (0,298)** et **margin_up (0,121)** : contributions importantes et complémentaires.
 - **height_right (0,054)**, **height_left (0,033)** : rôle secondaire
 - **diagonal (0,008)** quasi négligeable dans ce modèle



Le Random Forest : conclusion

- Performance : généralisation solide avec :
 - tout en classant parfaitement le train (100 %)
 - 99 % d'accuracy en test (3 erreurs / 300 : 2 faux acceptés, 1 vrai rejeté)
- Modèle : la Random Forest agrège de nombreux arbres via bootstrap et moyenne leurs prédictions, ce qui améliore la précision et limite l'overfitting
- Variables clés : l'importance Gini met clairement en avant length (0,486), puis margin_low et margin_up.

Résultats des différentes analyses

Modèle	Type	Accuracy (train)	Accuracy (test)	Erreurs (test)	Faux billets acceptés (billet faux prédit "vrai")	Vrais billets rejetés (billet vrai prédit "faux")
Régression logistique	Supervisé	0,993	0,990	3	2	1
KNN	Supervisé	1,000	0,990	3	2	1
Random Forest	Supervisé	1,000	0,990	3	2	1
K-Means	Non supervisé	0,985	0,987	4	2	2

Conclusion

4ème position : K-Means

Utile pour la segmentation/détection exploratoire, pas pour une décision binaire étiquetée sans post-mapping.

3ème position : KNN : à réserver au benchmark

Efficace mais sensible au scaling et plus coûteux à l'inférence (distance à de nombreux points) ; moins pratique en production à grande échelle.

Conclusion

2nd position : **Random Forest**

Comme baseline/backup, robuste sur données tabulaires, capte non-linéarités/interactions, peu de pré-traitements et scores proba via `predict_proba`. Excellent compromis précision/robustesse pour un filtre “vrai vs faux”.

Recommandation : Régression logistique

Simple, rapide, explicable, probabilités souvent bien calibrées (puis ajustables si besoin). Idéale en modèle de référence et pour le monitoring.

Merci pour votre attention

